



Модуль 5. Зависимости (Dependency Injection)

Введение в модуль

В Модуле 4 мы построили endpoint'ы, научились управлять жизненным циклом приложения через `lifespan` и хранить глобальное состояние в `app.state`. Но есть проблема: если каждый endpoint напрямую обращается к `app.state.db` или создаёт соединение с базой данных вручную, код превращается в спагетти, повторяется, и тестировать его невозможно.

Dependency Injection (DI) — это техника, при которой объект не создаёт свои зависимости сам, а получает их извне. FastAPI встроил DI на уровне фреймворка через механизм `Depends`. Это позволяет:

- разделять код на слои (бизнес-логика, доступ к данным, конфигурация);
- переиспользовать ресурсы (одно соединение с БД на запрос);
- тестировать компоненты изолированно, подменяя зависимости моками;
- управлять жизненным циклом ресурсов через `yield` (вспоминаем Модуль 1: генераторы и контекстные менеджеры).

В этом модуле мы пойдём от философии инверсии управления до конкретных паттернов проектирования, которые делают FastAPI-приложение масштабируемым.

5.1. Концепция внедрения зависимостей

5.1.1. Проблема: жёсткая связанность

Рассмотрим endpoint, который напрямую создаёт зависимости:

```
In [ ]: @app.get("/users/{user_id}")
        async def get_user(user_id: int):
            # Жёстко зашито: конкретный класс, конкретные настройки
            db = AsyncPostgresConnection(host="localhost", port=5432)
```

```

await db.connect()

user = await db.fetchrow("SELECT * FROM users WHERE id = $1", user_id)

await db.close()
return dict(user)

```

Проблемы:

1. **Невозможно протестировать** без настоящей базы данных.
2. **Дублирование**: код подключения к БД копируется в каждый endpoint.
3. **Невозможно сменить реализацию**: заменить PostgreSQL на SQLite для тестов — придётся менять каждый endpoint.
4. **Нет управления ресурсами**: если `fetchrow` выбросит исключение, `db.close()` может не выполниться.

5.1.2. Инверсия управления (IoC)

Голливудский принцип (Hollywood Principle): «Не звоните нам, мы сами вам позвоним» (Don't call us, we'll call you).

В традиционном коде бизнес-логика **сама** создаёт и вызывает инфраструктуру:

In []: Endpoint -> создаёт Database -> создаёт Connection -> выполняет запрос

В IoC фреймворк **предоставляет** инфраструктуру бизнес-логике:

In []: Framework -> создаёт Database -> передаёт в Endpoint -> Endpoint использует

Управление перевернуто: не бизнес-логика контролирует инфраструктуру, а инфраструктура «подставляет» себя в бизнес-логику.

5.1.3. Три способа внедрения зависимостей

Способ	Описание	Пример
Через конструктор	Зависимости передаются при создании объекта	<code>service = UserService(repo)</code>
Через сеттер	Зависимости устанавливаются после создания	<code>service.set_repo(repo)</code>
Через интерфейс	Зависимости передаются в метод при вызове	<code>def method(self, repo)</code>

FastAPI использует **вариант через интерфейс** (параметры функции), но автоматизирует его: фреймворк анализирует сигнатуру endpoint'a, видит аннотацию `Depends(...)`, вызывает зависимость и передаёт результат в функцию.

5.1.4. Математическая подоплека: граф зависимостей и топологическая сортировка

Представьте систему зависимостей как **ориентированный ациклический граф** (DAG, Directed Acyclic Graph):

- **Вершины** — компоненты (endpoint'ы, сервисы, репозитории, соединения с БД).
- **Рёбра** — зависимости: стрелка $A \rightarrow B$ означает « A зависит от B ».

Пример:

```
In [ ]: get_user(endpoint)
        └─ user_service
            └─ user_repository
                └─ database_connection
            └─ cache_client
                └─ redis_connection
```

FastAPI при обработке запроса должен **разрешить** этот граф: создать все зависимости в правильном порядке. Поскольку граф ацикличен, существует **топологическая сортировка** — порядок вершин, при котором каждая вершина идёт после всех своих зависимостей.

Алгоритм топологической сортировки (упрощённо, DFS-based):

```
In [ ]: function resolve(node):
        if node in resolved:
            return resolved[node]
        for dependency in node.dependencies:
            resolve(dependency)
        resolved[node] = create(node)
        return resolved[node]
```

FastAPI делает именно это: рекурсивно вызывает зависимости, кэширует результат (`use_cache=True` по умолчанию), и только потом вызывает endpoint.

Почему граф должен быть ацикличным? Если A зависит от B , а B зависит от A — получается цикл. FastAPI обнаружит это во время выполнения и выбросит `RecursionError`. Это соответствует математическому факту: топологическая сортировка существует тогда и только тогда, когда граф не содержит циклов.

5.2. Система Depends в FastAPI

5.2.1. Базовый синтаксис

```
In [ ]: from fastapi import Depends, FastAPI

app = FastAPI()

async def common_parameters(q: str | None = None, skip: int = 0, limit: int = 100):
    return {"q": q, "skip": skip, "limit": limit}

@app.get("/items/")
async def read_items(common_parameters: dict = Depends(common_parameters)):
    return common_parameters

@app.get("/users/")
async def read_users(common_parameters: dict = Depends(common_parameters)):
    return common_parameters
```

Что происходит:

1. FastAPI видит параметр `common_parameters: dict = Depends(common_parameters)`.
2. Вызывает `common_parameters(q=..., skip=..., limit=...)`, передавая query-параметры из запроса.
3. Результат (dict) передаётся в `read_items` как аргумент `common_parameters`.

Важно: `Depends` не создаёт класс, не вызывает конструктор. Это **декларатор зависимости**. Сама функция `common_parameters` — обычная функция, она может быть `async` или синхронной (FastAPI вызовет её через `run_in_threadpool`, если синхронная).

5.2.2. Классы-зависимости

Если зависимость требует состояния или нескольких методов, удобнее использовать класс:

```
In [ ]: class PaginationParams:
        def __init__(self, skip: int = 0, limit: int = 100):
            self.skip = skip
            self.limit = limit

@app.get("/items/")
async def read_items(pagination: PaginationParams = Depends()):
    return {"skip": pagination.skip, "limit": pagination.limit}
```

Когда `Depends()` вызывается без аргументов с классом, FastAPI создаёт экземпляр класса, передавая параметры в конструктор. Это эквивалентно `Depends(PaginationParams)`.

5.2.3. Вложенные зависимости

Зависимости могут зависеть от других зависимостей:

```
In [ ]: async def get_query_token(token: str):
        return token

async def get_token_header(token: str = Depends(get_query_token)):
    return {"token": token}

@app.get("/items/")
async def read_items(info: dict = Depends(get_token_header)):
    return info
```

FastAPI разрешает дерево рекурсивно:

```
In [ ]: read_items
        |
        +-- get_token_header
             |
             +-- get_query_token
```

Каждая зависимость вызывается **один раз на запрос** (если не отключено кэширование).

5.2.4. Кэширование зависимостей (`use_cache`)

По умолчанию `Depends` кэширует результат в рамках **одного запроса**:

```
In [ ]: async def get_db():
        db = AsyncSessionLocal()
        try:
            yield db
        finally:
            await db.close()
```

```
@app.get("/items/")
async def read_items(
    db1 = Depends(get_db),
    db2 = Depends(get_db),  # <- тот же объект, что и db1!
):
    assert db1 is db2  # True
```

Это критично для ресурсов вроде соединений с БД: если бы `get_db` вызывалась дважды, мы получили бы две разные сессии в одном запросе — и потеряли бы атомарность транзакции.

Отключение кэширования:

```
In [ ]: db = Depends(get_db, use_cache=False)
```

Используется редко, например, когда зависимость должна возвращать уникальный объект при каждом вызове (генератор ID, случайных чисел).

5.2.5. Управление жизненным циклом ресурсов: `yield`

Самый мощный паттерн DI в FastAPI — генераторы с `yield`. Мы уже видели это в Модуле 1 (контекстные менеджеры) и Модуле 4 (lifespan). Теперь применим к зависимостям:

```
In [ ]: from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker
        from sqlalchemy.orm import sessionmaker

        async_session = async_sessionmaker(engine, expire_on_commit=False)

        async def get_db() -> AsyncSession:
            async with async_session() as session:
                yield session
            # После yield выполняется при выходе из endpoint'a
            # Независимо от того, было исключение или нет
```

Что происходит под капотом:

1. FastAPI вызывает `get_db()`, получает генератор.
2. Вызывает `next()` (или `__anext__()` для `async`) — выполнение идёт до `yield session`.
3. `session` передаётся в endpoint.
4. Endpoint выполняется.
5. После завершения endpoint'a (успешного или с исключением) FastAPI возобновляет генератор.

6. Выполняется код после `yield` — закрытие сессии.

Это **асинхронный контекстный менеджер**, встроенный в систему DI. Он гарантирует, что ресурс будет освобождён.

Транзакции с `rollback` при ошибке:

```
In [ ]: async def get_db() -> AsyncSession:
        async with async_session() as session:
            try:
                yield session
                await session.commit()
            except Exception:
                await session.rollback()
                raise
            finally:
                await session.close()
```

Важное ограничение: внутри зависимости с `yield` можно выбросить исключение до `yield` — тогда endpoint не выполнится. Но если выбросить исключение **после** `yield`, оно заменит исключение из endpoint'a (или добавится как `__context__`). Обычно после `yield` пишут только `cleanup`-код, который не выбрасывает исключений.

5.2.6. Зависимости для целого роутера

Можно навесить зависимость на весь `APIRouter`:

```
In [ ]: from fastapi import APIRouter, Depends, HTTPException

        async def verify_token(x_token: str = Header()):
            if x_token != "secret-token":
                raise HTTPException(status_code=403, detail="Invalid token")

        router = APIRouter(
            prefix="/admin",
            dependencies=[Depends(verify_token)]
        )

        @router.get("/dashboard")
        async def admin_dashboard():
            return {"status": "ok"}

        @router.get("/stats")
        async def admin_stats():
            return {"users": 1000}
```

Все endpoint'ы в `router` автоматически требуют валидного `x_token`.

Это устраняет дублирование кода аутентификации.

5.2.7. Зависимости для всего приложения

```
In [ ]: app = FastAPI(dependencies=[Depends(verify_token)])
```

Теперь **все** endpoint'ы приложения требуют токен. Исключения можно сделать через `dependencies=[]` на отдельных роутерах или использовать более сложную логику внутри `verify_token`.

5.3. Практические паттерны DI

5.3.1. Паттерн «Repository»

Repository — абстракция, скрывающая детали доступа к данным. Бизнес-логика не знает о SQL, таблицах, ORM. Она работает с методами репозитория.

```
In [ ]: from typing import Protocol
        from pydantic import BaseModel

        class User(BaseModel):
            id: int
            name: str
            email: str

        class UserRepository(Protocol):
            async def get_by_id(self, user_id: int) -> User | None: ...
            async def create(self, user: User) -> User: ...

        class PostgresUserRepository:
            def __init__(self, session: AsyncSession):
                self._session = session

            async def get_by_id(self, user_id: int) -> User | None:
                result = await self._session.execute(
                    select(UserORM).where(UserORM.id == user_id)
                )
                orm_user = result.scalar_one_or_none()
                return User.model_validate(orm_user) if orm_user else None

            async def create(self, user: User) -> User:
                orm_user = UserORM(**user.model_dump())
                self._session.add(orm_user)
                await self._session.flush()
                return User.model_validate(orm_user)
```



```
# Зависимость
async def get_user_repo(session: AsyncSession = Depends(get_db)) -> UserRepository:
    return PostgresUserRepository(session)

@app.get("/users/{user_id}")
async def get_user(
    user_id: int,
    repo: UserRepository = Depends(get_user_repo)
):
    user = await repo.get_by_id(user_id)
    if not user:
        raise HTTPException(status_code=404, detail="User not found")
    return user
```

Преимущества:

- Можно создать `InMemoryUserRepository` для тестов.
- Смена PostgreSQL на MongoDB требует только новой реализации `UserRepository`.
- Бизнес-логика (endpoint) не зависит от SQLAlchemy.

5.3.2. Паттерн «Unit of Work»

Unit of Work (UoW) — паттерн, который группирует несколько операций в одну транзакцию. Все изменения фиксируются (commit) или откатываются (rollback) атомарно.

```
In [ ]: class UnitOfWork:
    def __init__(self, session: AsyncSession):
        self._session = session
        self.users = PostgresUserRepository(session)
        self.orders = PostgresOrderRepository(session)

    async def commit(self):
        await self._session.commit()

    async def rollback(self):
        await self._session.rollback()

async def get_uow(session: AsyncSession = Depends(get_db)) -> UnitOfWork:
    uow = UnitOfWork(session)
    try:
        yield uow
        await uow.commit()
    except Exception:
        await uow.rollback()
        raise

@app.post("/orders/")
async def create_order(
```

```

    order_data: OrderCreate,
    uow: UnitOfWork = Depends(get_uow)
):
    user = await uow.users.get_by_id(order_data.user_id)
    if not user:
        raise HTTPException(status_code=404)

    order = await uow.orders.create(order_data)
    # commit произойдёт автоматически при успешном выходе из endpoint'a
    return order

```

Почему это важно: если `uow.orders.create` выбросит исключение, `get_uow` автоматически вызовет `rollback`. База данных останется в согласованном состоянии.

5.3.3. Инъекция конфигурации

```

In [ ]: from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    database_url: str
    redis_url: str
    ml_model_path: str
    debug: bool = False

    class Config:
        env_file = ".env"

settings = Settings() # создаём один раз при импорте

def get_settings() -> Settings:
    return settings

@app.get("/health")
async def health(settings: Settings = Depends(get_settings)):
    return {"debug": settings.debug}

```

`get_settings` обернута в `lru_cache` или создаётся как singleton, чтобы не парсить `.env` на каждый запрос.

5.3.4. Инъекция логгера

```

In [ ]: import structlog
        from fastapi import Request

def get_logger(request: Request):
    return structlog.get_logger(
        request_id=request.headers.get("x-request-id", "unknown")
    )

```

```
@app.get("/items/")
async def read_items(logger = Depends(get_logger)):
    logger.info("Processing request")
    return {"items": []}
```

5.4. Тестирование зависимостей и Mocking

5.4.1. Проблема: как тестировать endpoint с зависимостями?

Если endpoint зависит от реального `get_db`, который подключается к PostgreSQL, тесты будут:

- медленными (сеть, диск);
- хрупкими (требуют запущенной БД);
- неизолрованными (тесты влияют друг на друга через общие данные).

Нужно подменить зависимости на тестовые реализации.

5.4.2. `app.dependency_overrides`

FastAPI предоставляет механизм **переопределения зависимостей** на уровне приложения:

```
In [ ]: from fastapi.testclient import TestClient

# Реальное приложение
from main import app, get_db, get_settings

# Фейковая зависимость
class FakeDB:
    async def fetch(self, query):
        return [{"id": 1, "name": "Test User"}]

def override_get_db():
    return FakeDB()

# Переопределяем
app.dependency_overrides[get_db] = override_get_db

client = TestClient(app)

def test_get_user():
    response = client.get("/users/1")
```

```
assert response.status_code == 200
assert response.json()["name"] == "Test User"
```

Как это работает:

`app.dependency_overrides` — это словарь {оригинальная_зависимость: замена}. Когда FastAPI разрешает граф зависимостей, он сначала проверяет этот словарь. Если зависимость переопределена — вызывается замена, а не оригинал.

5.4.3. Паттерн «Test Double»

В тестировании выделяют несколько типов «двойников»:

Тип	Описание	Пример
Dummy	Объект, который ничего не делает, нужен только для компиляции/типов	<code>Mock()</code> без настройки
Fake	Рабочая, но упрощённая реализация	<code>InMemoryRepository</code> вместо <code>PostgresRepository</code>
Stub	Возвращает заранее заданные ответы	<code>lambda: 42</code>
Spy	Запоминает, какие методы были вызваны	<code>Mock</code> с <code>assert_called_with</code>
Mock	Полностью контролируемый объект	<code>unittest.mock.Mock</code>

Fake-репозиторий:

```
In [ ]: class InMemoryUserRepository:
    def __init__(self):
        self._users = {}
        self._counter = 1

    async def get_by_id(self, user_id: int) -> User | None:
        return self._users.get(user_id)

    async def create(self, user: User) -> User:
        user.id = self._counter
        self._users[user.id] = user
        self._counter += 1
        return user

def override_get_user_repo():
    return InMemoryUserRepository()

app.dependency_overrides[get_user_repo] = override_get_user_repo
```

Mock через `unittest.mock`:

```
In [ ]: from unittest.mock import Mock, AsyncMock

def override_get_ml_model():
    mock = AsyncMock()
    mock.predict.return_value = [0.95, 0.05]
    return mock

app.dependency_overrides[get_ml_model] = override_get_ml_model
```

5.4.4. Pytest fixtures для DI

```
In [ ]: import pytest
from fastapi.testclient import TestClient
from main import app, get_db

@pytest.fixture
def fake_db():
    return FakeDB()

@pytest.fixture
def client(fake_db):
    app.dependency_overrides[get_db] = lambda: fake_db
    yield TestClient(app)
    app.dependency_overrides.clear() # ЧИСТИМ ПОСЛЕ ТЕСТА

def test_read_items(client):
    response = client.get("/items/")
    assert response.status_code == 200
```

Важно: `app.dependency_overrides.clear()` в `finally` или через `yield`-fixture. Иначе переопределения «просочатся» в другие тесты.

5.4.5. Тестирование асинхронных зависимостей

Если зависимость `async`, а `TestClient` синхронен — `dependency_overrides` должна возвращать **awaitable** или сам объект:

```
In [ ]: # Плохо: TestClient не умеет await'ить синхронную функцию, возвращающую к
def bad_override():
    return async_get_db() # <- возвращает корутину, а не объект!

# Хорошо: возвращаем готовый объект
async def good_override():
    return FakeDB()

app.dependency_overrides[get_db] = good_override
```

Или используйте `AsyncClient` из `httpx` для полностью асинхронных

тестов:

```
In [ ]: import pytest
        from httpx import AsyncClient

        @pytest.fixture
        async def async_client():
            async with AsyncClient(app=app, base_url="http://test") as client:
                yield client

        @pytest.mark.asyncio
        async def test_read_items(async_client):
            response = await async_client.get("/items/")
            assert response.status_code == 200
```

5.4.6. Математическая подоплека: подстановка в теории категорий

Переопределение зависимостей в тестах — это частный случай **подстановки** (substitution) в теории категорий.

Если у нас есть интерфейс (Protocol) I и две реализации A (реальная) и B (фейковая), обе с морфизмом $A \rightarrow I$ и $B \rightarrow I$, то любая функция $f: I \rightarrow X$, использующая интерфейс, может быть применена и к A , и к B .

Формально, если $B <: I$ и $A <: I$ (оба подтипы интерфейса), то:

$\forall f: I \rightarrow X, \quad f(A) \text{ корректно} \iff f(B) \text{ корректно}$

Это **принцип подстановки Барбары Лисков** (Liskov Substitution Principle): объекты подтипов должны быть взаимозаменяемы без нарушения корректности программы.

В контексте DI: если `PostgresUserRepository` и `InMemoryUserRepository` оба реализуют `UserRepository`, то endpoint, зависящий от `UserRepository`, не должен заметить замены.

Итог модуля 5

Концепция	Суть	Практическое применение
IoC	Инфраструктура предоставляет себя бизнес-логике, а не наоборот	Разделение ответственности
DI	Зависимости передаются извне, а не	Тестируемость,

Концепция	Суть	Практическое применение
	создаются внутри	гибкость
Depends	Механизм FastAPI для автоматического разрешения зависимостей	Уменьшение шаблонного кода
Вложенные зависимости	Дерево зависимостей, разрешаемое рекурсивно	Многоуровневая архитектура
Кэширование	<code>use_cache=True</code> — одна зависимость на запрос	Транзакции, сессии БД
Yield	Генераторы для setup/teardown в DI	Автоматическое закрытие ресурсов
Repository	Абстракция над источником данных	Смена БД без изменения endpoint'ов
Unit of Work	Атомарная группа операций	Целостность транзакций
dependency_overrides	Подмена зависимостей в тестах	Изолированное тестирование
Test Double	Fake, Mock, Stub для зависимостей	Быстрые и надёжные тесты

В **Модуле 6** мы применим DI к конкретной инфраструктуре: асинхронным базам данных. Разберём SQLAlchemy 2.0, подключение через Depends, управление транзакциями и миграции Alembic.